

The hackathon expectation



In this hackathon, you will design and build a **working prototype** using **IBM Bob IDE**, aligned to the following theme:

Build with purpose using IBM Bob 2.0

Create a solution that improves a specific developer workflow, such as onboarding, debugging, code review, testing, application maintenance, or release and deployment processes. Start by clearly defining a problem where time, effort, or errors are too high today. Then, using IBM Bob 2.0, build a working prototype on a real or sample project that demonstrates a full solution to improve the specified workflow. Leverage features like Agent mode, parallel tasks, subagents, and document understanding to manage and improve multiple steps, not just assist with coding. Clearly demonstrate impact by showing how your solution increases productivity, reduces manual effort, errors, and rework, or significantly shortens the time required to complete tasks.

Refer to [example use cases](#) to help you brainstorm ideas for building a solution.

⚠ Important: Participants are required to upload all relevant **Bob IDE task session summary screenshots** to their code repository as evidence of Bob usage. Learn [how to upload task session summary](#).

Participants may also **optionally** use the following watsonx products:

- **IBM watsonx Orchestrate** – a no-code and low-code platform designed to orchestrate AI agents across business workflows. With watsonx Orchestrate, you can create, deploy, and manage intelligent agents and assistants that automate tasks, streamline processes, and enhance productivity.
- **IBM watsonx.ai** – a powerful AI studio that supports the development of agentic AI solutions using a wide range of foundation models. Through the Prompt Lab, participants can experiment with IBM's Granite models and other leading models to create agents that understand and respond to natural language. IBM watsonx.ai can also serve as an inference provider for your agents, allowing them to generate responses, make decisions, and interact with users or systems intelligently.

A note on using other technologies

You may use any framework or technology to build your solution as long as you adhere to the product usage policies. However, to be eligible for judging, your solution must showcase **IBM Bob IDE** as a core component.

A note on data sets before you begin

Participants are required to bring their own datasets to build the solution aligning to your use case. As you collect data for your project, you'll want to use best practices. Here are helpful tips:

- Participants are responsible for ensuring data is compliant.
- Data from public websites may be used, if the terms allow for commercial use, but please keep a list of the websites you use.
- Do not use data or assets containing company confidential data, or any other data without permission from the data owner. Participants are responsible for getting approval.
- Do not use any client data.
- Do not use any data containing personal information (PI).
- Do not use data obtained from social media.

Set up IBM Bob account



To access and use the hackathon-provisioned IBM Bob account, participants must be registered for the hackathon.

⚠ Important: If you already have access to Bob through your personal account, please use the hackathon-provisioned Bob account for this event and to avoid consuming usage on your personal account.

Confirm your hackathon IBM Bob account

At the start of the hackathon, all registered members will receive an email invite to join the **hackathon-provisioned Bob account**. Follow the steps below to confirm your account:

1. Check the email inbox you used to register for the hackathon and open the email invite you received from the IBM Bob team about joining your hackathon-provisioned Bob account. To confirm you have the correct email, look for:
 - You have been added as a team member to **ibm-hackathon-xxxx**

i Note: The team member reference mentioned in the email does not mean that you have been added to a team for this hackathon. It is simply terminology used as part of the account enablement process through an Enterprise account. Participants who wish to join or form a team must do so through the supported hackathon platforms.

- **Bob Plan:** Enterprise plan

Please check your Junk/Spam folders if you cannot find the email in your inbox. You can also quickly search for “IBM Bob” to locate the email.

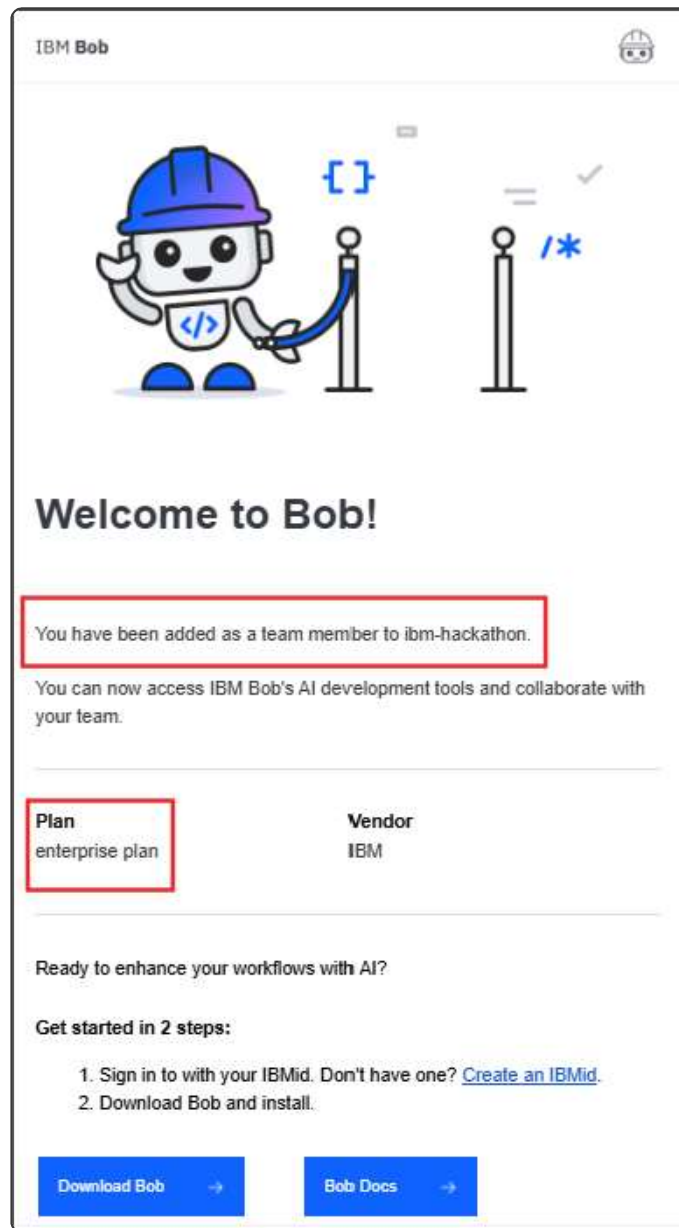


Figure 2: Screenshot or diagram

1. [Create an IBMid](#) if you have not created one yet. IBMid is required to access Bob.

Bobcoins

Each interaction with Bob's AI capabilities consumes Bobcoins. For this hackathon, **40 Bobcoins** will be automatically applied to your hackathon-provisioned **IBM Bob account**. These Bobcoins are intended to be sufficient for designing and building a compelling proof-of-concept submission.

It is recommended that you plan ahead with your teammates to divide tasks so that you can make full use of the total Bobcoins allocated across all team members. Once your Bobcoin usage reaches **100% usage**, **no additional Bobcoins will be provided** as part of the hackathon. You may still continue working on your

solution and final submission. For example, consider leveraging the optional watsonx technologies enabled for this hackathon.

Follow [Bob best practices](#) to use the tool effectively and efficiently.

You can monitor your Bobcoin usage in Bob IDE or [Admin dashboard](#):

- In your Bob IDE, select the **Settings** icon and the **Settings** tab will open.

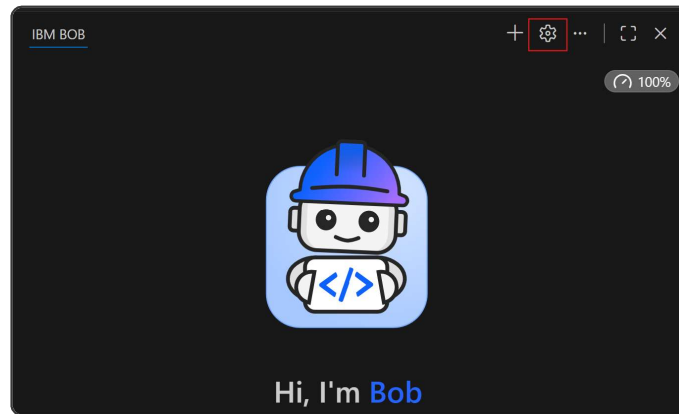


Figure 3: Screenshot or diagram

- Under the **General** section, you can view your Bobcoin usage. If you have access to multiple Bob accounts, ensure that you select the hackathon provisioned instance named **ibm- coding-challenge-xxx (region: us-east)**.

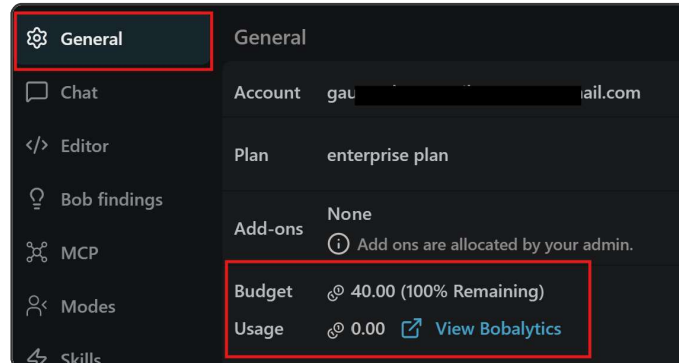


Figure 4: Screenshot or diagram

i Note: The team name reference mentioned in the settings does not mean that you have been added to a team for this hackathon. It is simply terminology used as part of the account enablement process through an Enterprise account. Participants who wish to join or form a team must do so through the supported hackathon platforms.

Working with IBM Bob

To participate in the hackathon, you must install and use **Bob IDE**. You may also optionally use **Bob Shell**.

1. [Bob IDE](#) (Required)
2. [Bob Shell](#) (Optional)

⚠ **Important:** You must use **Bob IDE** for this hackathon. As part of your submission, you will be required to upload your **Bob task session summary** as evidence of Bob usage.

1. Bob IDE

Install the Bob IDE, sign into your Bob hackathon account, and start building your solution with Bob.

Install Bob IDE

Follow the [Bob IDE installation instructions](#).

⚠ **Important:** IDE v1.0.3 and v2.0.0 will stop working on September 30, 2026.

- If you are on v1.0.3, you must upgrade to the latest v2.0.x release.
- If you are on v2.0.0, you must upgrade to v2.0.2 or later.

Sign into Bob IDE

To use Bob IDE after installing it, you have to sign in to Bob using your hackathon registration email.

⚠ **Important:** An IBMid is required to authenticate. If you do not have one, see [Create an IBMid](#) to create one for your hackathon registered email.

- Open your Bob IDE and select the **Log in to Bob** button on the IDE.

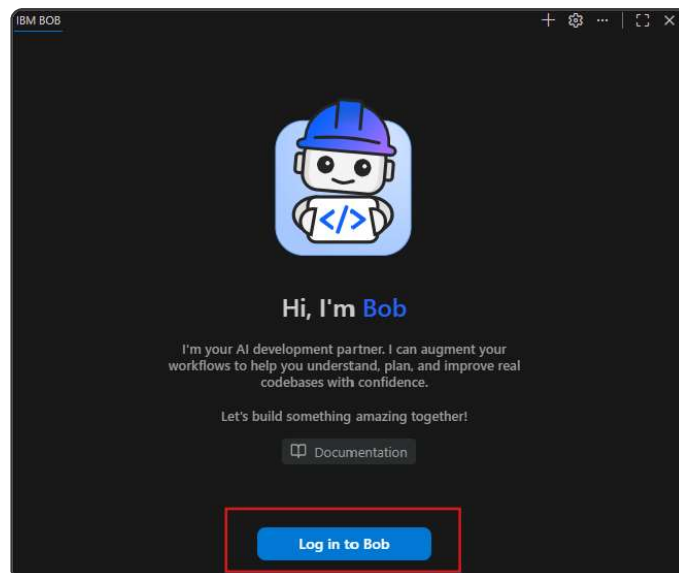


Figure 5: Screenshot or diagram

- Follow and complete the IBMid authentication process on your browser. Once authentication is complete, Bob chat interface will be displayed for you to get started.

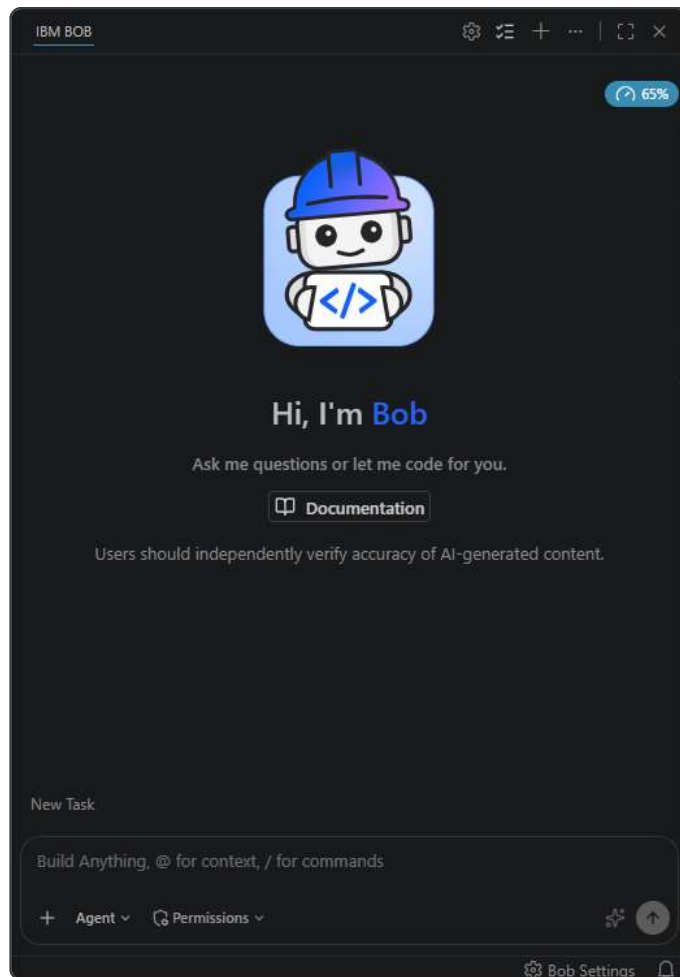


Figure 6: Screenshot or diagram

- **Important:** If you have access to multiple Bob accounts, ensure that you select the hackathon provisioned instance named **ibm- coding-challenge-uat (region: us-east)** to avoid consuming Bobcoins from your personal account. To switch to hackathon-provisioned account, select the **Settings** icon in the Bob IDE and it will open the Settings tab. Under General, you can switch the team to **ibm- coding-challenge- uat (region: us-east)** instance.

i Note: The team name reference mentioned in the settings does not mean that you have been added to a team for this hackathon. It is simply terminology used as part of the account enablement process through an Enterprise account. Participants who wish to join or form a team must do so through the supported hackathon platforms.

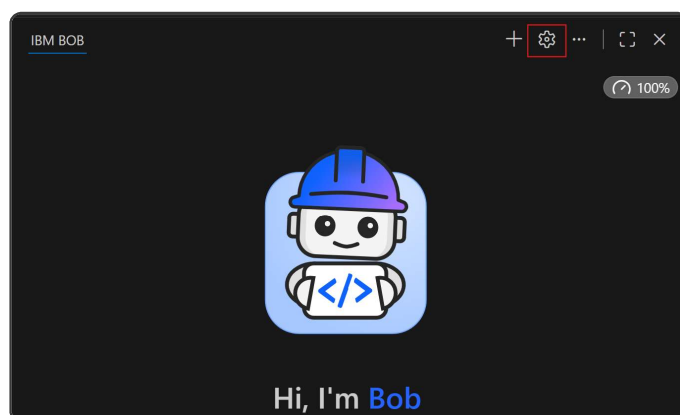


Figure 7: Screenshot or diagram

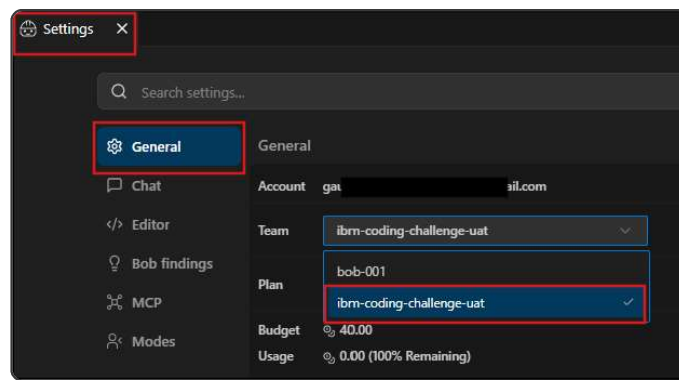


Figure 8: Screenshot or diagram

Note: If you experience network issues with outbound traffic, you might need to configure your firewall settings. For more information, see [Configuring firewall rules for Bob](#).

Best practices

Follow the [best practices guidelines](#) to get the most out of your experience with Bob.

Bob IDE features and configuration

Explore Bob IDE features and their configuration to enhance your experience working with Bob and solution building.

- [Chat interface](#)

Bob IDE's chat interface is your primary workspace where you use natural language prompts, slash commands, and auto-approval for AI-assisted coding workflows. Learn more about [the chat interface and its components](#).

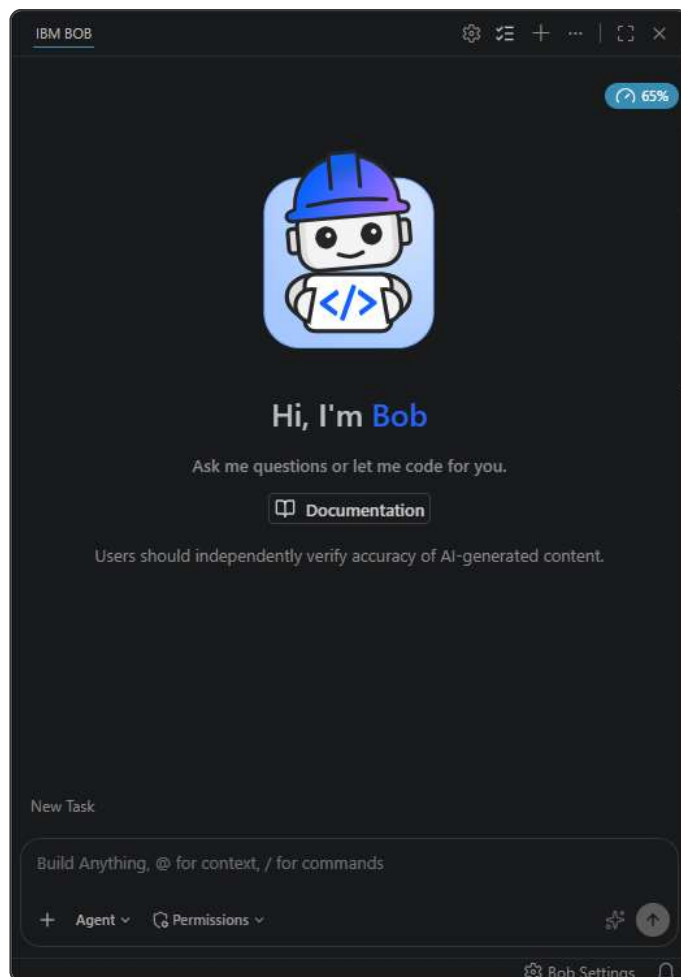


Figure 9: Screenshot or diagram

- [Modes](#)

Modes are specialized personas that tailor Bob's behavior for your specific tasks. Each mode offers different capabilities and access levels to help you accomplish particular goals more efficiently. Learn more about [modes and their usage](#).

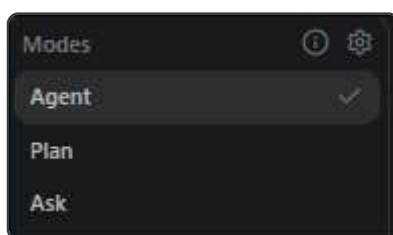


Figure 10: Screenshot or diagram

- [Custom modes](#)

Tailor Bob's behavior for specific tasks or workflows by creating custom modes. You can make modes global (available across all projects) or project-specific. Learn more about [subagents](#).

- [Subagents](#)

Learn how Bob uses subagents to handle focused, self-contained tasks in isolated contexts without polluting the main conversation. Learn more about [custom modes and its configuration](#).

- [Auto-approve](#)

Auto-approve actions in Bob IDE eliminate repetitive prompts but increase security risks. Learn safe configuration practices for read, write, and execute permissions. Learn more about [auto-approve and its configuration](#).

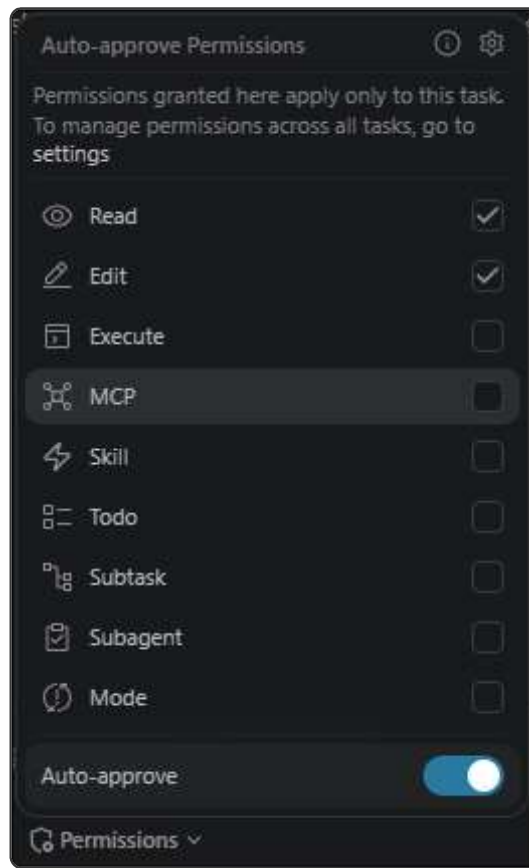


Figure 11: Screenshot or diagram

- [Custom rules](#)

Custom rules influence how Bob responds to your requests, aligning output with your specific preferences and project requirements. You can control coding style, documentation approach, and decision-making processes. Learn more about [custom rules and its configuration](#).

- [MCP servers](#)

The Model Context Protocol (MCP) extends Bob's capabilities by connecting to external tools and services. This guide shows you how to configure, manage, and use MCP servers with Bob. Learn more about [MCP server configuration with Bob](#).

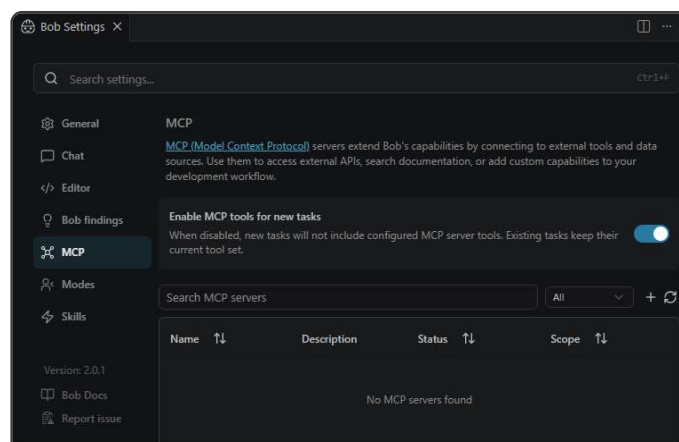


Figure 12: Screenshot or diagram

- [Ignoring files](#)

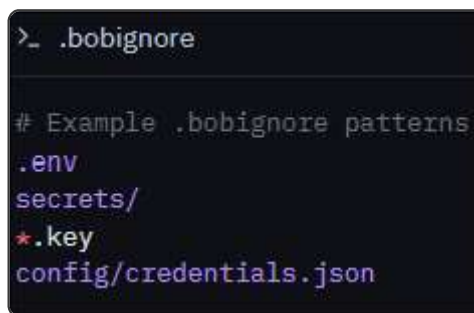
Control which files Bob can access by creating a `.bobignore` file in your project. Learn more about [Ignoring files](#).

- [Context mentions](#)

Context mentions let you reference specific elements of your project directly in your conversations with Bob. By using the @ symbol, you can point Bob to files, folders, problems, and other project components, enabling more accurate and efficient assistance. Learn more about [context mentions](#).

- [Security guidelines](#)

Bob has capabilities for coding and system interaction. To use safely, follow the [security guidelines](#).



```
> .bobignore

# Example .bobignore patterns
.env
secrets/
*.key
config/credentials.json
```

Figure 13: Screenshot or diagram

- [Bob tips](#)

Bob tips detects code quality issues in real-time with AI-powered refactoring suggestions for complex functions. Reduce technical debt as you code. Learn more about [Bob tips and findings](#).

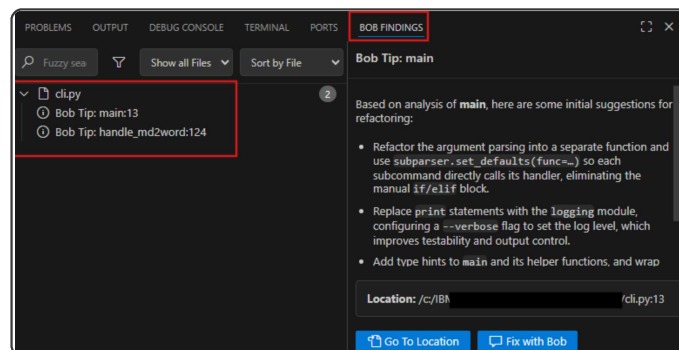


Figure 14: Screenshot or diagram

- [Rollback](#)

Automatically version workspace files during AI tasks. Safely explore AI suggestions and experiment with changes, and easily recover from unwanted changes. Learn more about [Rollback](#).

- [Code actions](#)

Access Bob's AI-powered code actions that provide quick fixes, refactorings, and code explanations directly in the Bob IDE through lightbulb icons and context menus. Learn more about [utilizing code actions](#).

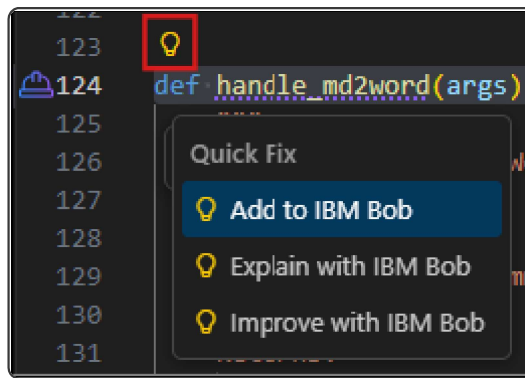


Figure 15: Screenshot or diagram

- [Code reviews](#)

Use the built-in Review workflow to get AI-powered code reviews directly in your IDE. Bob analyzes changes and flags potential issues before you commit your work. Learn more about [code reviews and its usage](#).

Quick Reference Card	
Command	Use Case
<code>/review</code>	Quick code check
<code>/review --branch main</code>	Pre-PR review
<code>/review #123 --issue-coverage</code>	Verify issue completion
<code>/review --pr-desc</code>	Generate PR description
<code>/review file.py</code>	Focused file review

Figure 16: Screenshot or diagram

- [Commit messages](#)

Generate meaningful commit messages automatically with Bob. Save time, maintain consistency, and follow conventional commit standards with AI assistance. Learn more about [commit message and its usage](#).

- [Pull requests](#)

Bob can generate pull requests (PRs) and PR descriptions directly from your IDE, streamlining your development workflow and saving time. Learn more about [pull requests and its usage](#).

- [Enhance prompt](#)

Improve your prompts before sending them to Bob. Click the sparkles icon in the chat input to automatically refine your request, making it clearer, more specific, and more likely to yield better results. Learn more about [enhancing prompts](#).

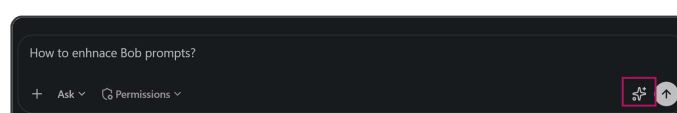


Figure 17: Screenshot or diagram

- [Literate coding](#)

Write code with AI assistance directly in your editor using natural language instructions right where the code should go. Bob generates the implementation in context with inline diffs. Learn more about [literate coding and its usage](#).

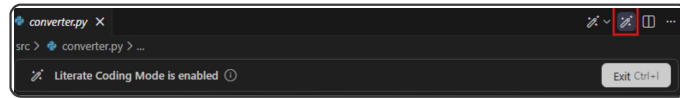


Figure 18: Screenshot or diagram

- [Skills](#)

Create reusable instruction sets for specialized workflows. Define custom workflows, add supporting files, and teach Bob specialized tasks for consistent results. Learn more about [skills and its usage](#).

Bob IDE hands-on exercises

Try the quick start hands-on exercises for sample use cases to get started with using Bob.

⚠ Important: You must use the [hackathon-provisioned IBM Bob account](#) to avoid usage on your personal Bob account, if you have one.

- [Bob exercises](#)

Quick hands-on exercises to learn how to use IBM Bob in the IDE for understanding code, refactoring, and automating everyday development tasks.

- [Bob and watsonx Orchestrate exercises](#)

Hands on exercises that show how IBM Bob and watsonx Orchestrate work together to design, build, and run agent-based workflows.

Bob exercises:

- [Quickstart exercise](#)

Use IBM Bob to build a web UI for an existing Node.js Express API and run it in Docker, using modes, the approval workflow, and agentic iteration.

- [Travel demo app](#)

Get started with IBM Bob and the Galaxium Travels demo app by learning the tutorial flow, setup requirements, and application architecture.

- [Build agents with Bob /init](#)

Give Bob persistent project context across conversations and modes with /init, which automatically generates AGENTS.md files.

- [Create a commit and pull request with Bob](#)

Use Bob to create a branch, stage changes, generate a commit message, push to your repository, and open a pull request.

- [Generate code from comments](#)

Use literate coding to have Bob generate code from natural language comments, where Bob helps you write precise, context-aware code modifications directly in your editor.

- [Plan and implement complex features](#)

Learn how to use the agentic chat sidebar to employ a Plan mode and Code mode workflow, where Bob plans complex features end-to-end and then implements them autonomously across your full codebase.

- [Standardize Bob's behavior](#)

Standardize Bob's behavior across your team using project-level rules files that tell Bob to document its code and remember its previous actions.

- [Add a custom mode](#)

Extend Bob's job capabilities by creating a custom product-manager mode with a tailored role definition, behavioral instructions, and deterministic tool access constraints.

- [Create a new context window](#)

Manage Bob's context window to preserve memory, control cost, and maintain output quality during complex or long-running conversations.

- [Modernize a Node.js application](#)

Learn to use IBM Bob for application modernization by upgrading a Node.js Express API from version 16 to 22. Try AI-assisted development with modes, approvals, and literate coding in this hands-on tutorial.

- [Inspect an unfamiliar codebase](#)

Use IBM Bob to rapidly understand an unfamiliar application, such as its purpose, project structure, architecture, tech stack, key components, test coverage, and deployment model. You can do this without relying on outdated documentation or waiting for teammates.

- [Generate architecture diagrams](#)

Use IBM Bob to analyze the Galaxium Travels codebase and generate Mermaid UML class diagrams, sequence diagrams, and use case diagrams. Learn how to use context mentions in Ask mode to explore code and Agent mode to save the results to your repository.

- [Audit code and generate reports](#)

Use IBM Bob to create a reusable security audit skill, scan an application against OWASP ASVS requirements, and generate SARIF and OSCAL reports that developers and AI agents can act on.

- [Generate secure code with an actor-critic workflow](#)

Use IBM Bob to configure security rules and apply an actor-critic pattern to generate Python code that satisfies security frameworks before it reaches a static analysis tool.

Bob and watsonx Orchestrate exercises:

- [Using IBM Bob to build watsonx Orchestrate agents and MCP tools](#)

A hands-on guide for automating the full process of building and deploying an agentic workflow using IBM Bob.

- [Build agentic workflows programmatically on watsonx Orchestrate using IBM Bob](#)

A hands-on guide for creating automated invoice-processing agentic workflows using IBM Bob to generate code, tools, and configuration for watsonx Orchestrate.

- [Turn BPMN diagrams into production-ready agents with Bob skills and watsonx Orchestrate](#)

A hands-on guide for using Bob skills to transform BPMN process models into SOP-driven, fully tested, and deployable watsonx Orchestrate agents with best-practice workflows, tools, and automation patterns.

2. Bob Shell (Optional)

Bob Shell brings IBM Bob's AI capabilities to your command line. Bob Shell delivers the same context awareness and reasoning-focused approach from IBM Bob but optimized for shell environments and automated processes.

Install Bob Shell

Follow the [Bob Shell installation instructions](#).

Important: Upgrading from Bob Shell 1.0.x

Bob Shell 2.0.0 requires a fresh install. Currently there is no automated upgrade path from 1.0.x. Your existing settings and configurations will be preserved during the install.

Authentication

Bob Shell uses the same login entry point as all other Bob clients: bob.ibm.com/login. Learn more about [Bob Shell authentication](#).

Start an interactive session

Interactive sessions provide a conversational interface to Bob directly in your terminal, allowing real-time assistance with your development tasks. Learn more about [starting interactive sessions](#).



Figure 19: Screenshot or diagram

Start a non-interactive session

Non-interactive session provides a method to use Bob Shell directly from the command line without entering an interactive session. Use for automation, scripting, and batch processing tasks. Learn more about [starting non-interactive sessions](#).

Bob Shell configuration

You can configure Bob Shell to match your workflow preferences. Learn more about [Bob Shell configuration](#).

Bob Shell tools

Learn how Bob Shell uses specialized tools to read files, edit code, run commands, and interact with your development environment from the command line. Learn more about [Bob Shell tools](#).

Bob Shell features

Explore Bob Shell features to enhance your experience and solution building.

- [Slash commands](#)

Create custom slash commands to automate workflows and standardize team practices. Learn more about [Slash commands](#).

- [Modes](#)

Modes are specialized personas that tailor Bob's behavior for your specific tasks. Each mode offers different capabilities and access levels to help you accomplish particular goals more efficiently. Learn more about [Modes](#).

Bob Shell usage examples

Try practical examples on how to use Bob Shell for debugging, code improvement, file creation, documentation generation, and learning new concepts. Learn more about [Bob Shell usage examples](#).

Upload Bob task session summary

Each participant must upload all relevant Bob task session summary screenshots as part of your project submission. These screenshots must be included in your **final code repository** as **submission deliverable**.

Follow the steps below to complete this process:

- Create a folder named **bob_sessions** in your project submission code repository.
- In your Bob IDE's chat interface, select **Tasks** to display the task list.

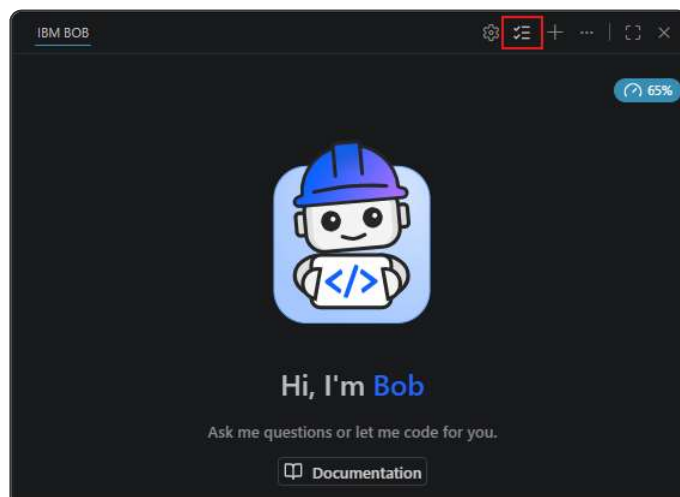


Figure 20: Screenshot or diagram

- Select a task from the list related to your project submission and the selected task will open in the chat panel. Confirm that you are in the correct project workspace. If your submission includes tasks from multiple workspaces, select **All** to view tasks across all relevant workspaces.

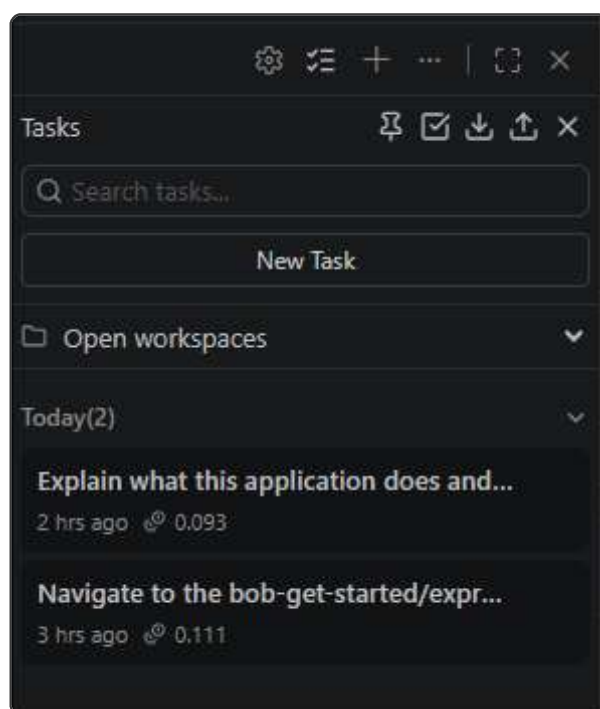


Figure 21: Screenshot or diagram

- Select the task header. A task session consumption summary will be displayed.

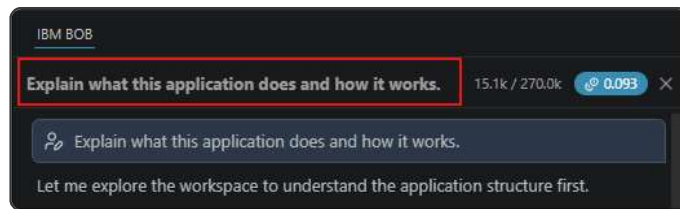


Figure 22: Screenshot or diagram

- Take a screenshot of the task session consumption summary. Save screenshots in **PNG** format where possible for better text clarity. Use a clear file name that includes your team name, task number, and short task description.

Example: teamalpha_task01_login_flow_summary.png

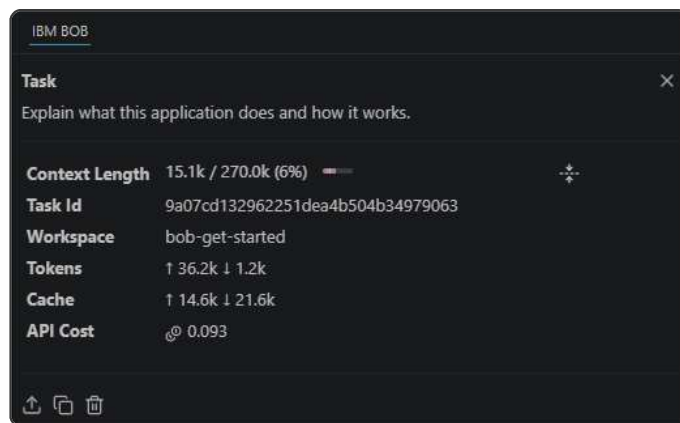


Figure 23: Screenshot or diagram

- Now, repeat the steps above for all the tasks related to your project submission.
- Upload all task session consumption summary screenshots to the **bob_sessions** folder in your code repository.

⚠ Important: Before submitting your project, ensure that your team final code repository includes the **bob_sessions** folder with all required screenshots relevant to the project solution.

Appendix: Example use cases

You are not limited to these ideas, but here are several examples for how you could apply IBM Bob, with optional use of watsonx Orchestrate and watsonx.ai, to solve real challenges in this hackathon:

- **Smart developer onboarding assistant:** Help new developers understand and contribute to an unfamiliar codebase faster. Use IBM Bob to analyze repositories, explain architecture, generate setup guidance, and suggest starter tasks. Optionally use watsonx Orchestrate to automate onboarding activities and watsonx.ai to deliver conversational guidance.
- **Intelligent code review and quality coach:** Improve code quality and reduce review effort by using IBM Bob to analyze code changes, identify risks, explain findings, recommend fixes, and generate

review summaries. Optionally use watsonx.ai to prioritize findings and watsonx Orchestrate to automate review workflows.

- **Automated testing and validation hub:** Reduce manual testing effort by using IBM Bob to generate unit tests, identify coverage gaps, validate code changes, and suggest improvements. Optionally use watsonx Orchestrate to orchestrate testing workflows and watsonx.ai to analyze test results and risk areas.
- **Release readiness and deployment assistant:** Help teams prepare software releases with confidence. Use IBM Bob to analyze code changes, review dependencies, summarize risks, generate release notes, and validate deployment requirements. Optionally use watsonx Orchestrate to automate release tasks and approvals.
- **Legacy application modernization accelerator:** Help teams understand, document, refactor, and modernize legacy applications. Use IBM Bob to explain existing code, identify modernization opportunities, generate updated components, and reduce migration effort. Optionally use watsonx.ai for impact analysis and watsonx Orchestrate to coordinate modernization workflows.